

Lý thuyết chuyên sâu: Kỹ thuật Hai con trỏ & Cửa sổ trượt (Two Pointers & Sliding Window)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)
Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool
Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

1. Bản chất toán học & Điều kiện áp dụng

1.1. Tính chất đơn điệu (Monotonicity Property)

Kỹ thuật **Hai con trỏ (Two Pointers)** là một kỹ thuật tối ưu hóa giải thuật, cho phép giảm độ phức tạp thời gian từ duyệt vét cạn $O(N^2)$ xuống $O(N)$ **tuyến tính**.

🔑 **Điều kiện tiên quyết để áp dụng:**
Bài toán phải thỏa mãn **Tính chất đơn điệu (Monotonicity)**:
Khi con trỏ thứ hai (R) di chuyển theo một chiều (ví dụ: tăng dần), vị trí tối ưu của con trỏ thứ nhất (L) cũng **chỉ di chuyển theo một chiều duy nhất** (không bao giờ phải quay lùi lại).

Nếu tính chất này bị vi phạm (ví dụ: mảng có cả số âm và số dương trong bài toán tính tổng), con trỏ L có thể phải nhảy tới nhảy lui, khi đó kỹ thuật Hai con trỏ sẽ **thất bại** (cần đổi sang Prefix Sum + Hash Map hoặc Quy hoạch động).

1.2. Chứng minh độ phức tạp bằng Phân tích khấu hao (Amortized Analysis)

Nhiều lập trình viên mới bắt đầu thường nhầm lẫn rằng hai vòng lặp lồng nhau có dạng:

```
int L = 0;
for (int R = 0; R < n; ++R) {
    while (condition) {
        L++;
    }
}
```

sẽ tốn $O(N^2)$ thời gian. **Điều này hoàn toàn sai trong kỹ thuật Hai con trỏ!**

Chứng minh:

- Con trỏ R bắt đầu từ 0 và tăng dần đến $N - 1$: Thực hiện đúng N bước tăng.
- Con trỏ L bắt đầu từ 0 và **chỉ tăng**, không bao giờ bị gán lùi lại giá trị cũ.

3. Trong suốt toàn bộ quá trình chạy của chương trình, L chỉ có thể tăng tối đa từ 0 lên N : Vòng lặp `while` chỉ có thể được thực thi **tối đa N lần trên toàn bộ chương trình**.

4. Tổng số bước di chuyển của cả hai con trỏ là:

$$\text{Total Steps} = (\text{Số bước của } R) + (\text{Số bước của } L) \leq N + N = 2N$$

5. Do đó, chi phí trung bình (khấu hao) cho mỗi bước của vòng ngoài là $O(1)$.

Tổng độ phức tạp thời gian đạt chính xác $O(N)$!

2. Phân loại 3 Mô hình Hai con trỏ Kinh điển trong CP

2.1. Mô hình 1: Hai con trỏ ngược chiều (Đối đầu / Converging Pointers)

Điều kiện & Cách hoạt động:

- Áp dụng trên **mảng đã sắp xếp tăng dần** $A_1 \leq A_2 \leq \dots \leq A_N$.
- Khởi tạo: Một con trỏ ở đầu mảng ($L = 0$), một con trỏ ở cuối mảng ($R = N - 1$).
- Tại mỗi bước, ta so sánh tổng $S = A[L] + A[R]$ với giá trị mục tiêu `Target`:
- Nếu $S == \text{Target}$: Đã tìm thấy nghiệm.
- Nếu $S < \text{Target}$: Vì mảng đã sắp xếp, để tăng tổng lên ta **bắt buộc phải tăng L** ($L \leftarrow L + 1$).
- Nếu $S > \text{Target}$: Để giảm tổng xuống ta **bắt buộc phải giảm R** ($R \leftarrow R - 1$).
- Điều kiện dừng: Khi hai con trỏ gặp nhau ($L \geq R$).

Độ phức tạp:

$O(N \log N)$ cho bước sắp xếp mảng + $O(N)$ cho bước duyệt hai con trỏ $\implies O(N \log N)$ **tổng thể**.

2.2. Mô hình 2: Hai con trỏ cùng chiều (Cửa sổ trượt / Caterpillar Method)

Điều kiện & Cách hoạt động:

- Hai con trỏ cùng xuất phát từ đầu mảng và cùng di chuyển về bên phải ($L \leq R$).
 - Đoạn $[L, R]$ tạo thành một **cửa sổ (window)** linh hoạt có kích thước co giãn:
 - **Mở rộng (Expand)**: Cho R tăng dần để nạp thêm phần tử $A[R]$ vào cửa sổ và cập nhật trạng thái cửa sổ.
 - **Thu hẹp (Shrink)**: Trong khi trạng thái trong cửa sổ vi phạm ràng buộc đề bài, ta loại bỏ phần tử $A[L]$ ra khỏi cửa sổ và tăng L lên cho đến khi cửa sổ hợp lệ trở lại.
 - **Cập nhật đáp án**: Ghi nhận độ dài lớn nhất, nhỏ nhất hoặc số lượng đoạn con hợp lệ.
-

2.3. Mô hình 3: Hai con trỏ trên 2 dãy riêng biệt (Two Sequences Merge)

Điều kiện & Cách hoạt động:

- Cho 2 dãy số đã được sắp xếp: Dãy A (độ dài N , con trỏ i) và dãy B (độ dài M , con trỏ j).
- Khởi tạo $i = 0, j = 0$.
- Tại mỗi bước, so sánh $A[i]$ và $B[j]$:
- Nếu $A[i] < B[j] \implies$ Xử lý $A[i]$ và tăng $i++$.
- Nếu $A[i] > B[j] \implies$ Xử lý $B[j]$ và tăng $j++$.
- Nếu $A[i] == B[j] \implies$ Ghi nhận phần tử chung (giao của 2 tập hợp) và tăng cả $i++, j++$.
- **Độ phức tạp:** $O(N + M)$ thời gian, $O(1)$ bộ nhớ phụ.

3. Kỹ thuật Đếm đoạn con (Counting Subarrays)

3.1. Kỹ thuật đếm số đoạn con kết thúc tại R

Xét bài toán: "Đếm số đoạn con liên tiếp có tổng $\leq K$ trong mảng các số không âm".

- **Tính chất:** Nếu đoạn $[L, R]$ có tổng $\leq K$, thì **mọi đoạn con kết thúc tại R và bắt đầu từ l' ($L \leq l' \leq R$)** cũng chắc chắn có tổng $\leq K$ (vì các phần tử đều không âm).
- Các đoạn con hợp lệ kết thúc tại R gồm:

$$[L, R], [L + 1, R], [L + 2, R], \dots, [R, R]$$

- **Số lượng đoạn con hợp lệ kết thúc tại R là:**

$$\Delta = R - L + 1$$

- **Tổng số đoạn con hợp lệ toàn mảng:**

$$\text{Total} = \sum_{R=0}^{N-1} (R - L_R + 1)$$

3.2. Kỹ thuật "Chính xác K " quy về hiệu của hai bài toán "Nhiều nhất K "

Nhiều bài toán yêu cầu:

"Đếm số đoạn con có chính xác K phần tử phân biệt" hoặc "Đếm số đoạn con có tổng chính xác bằng K ".

Thách thức:

Điều kiện "chính xác K " **không có tính đơn điệu**: Khi R tăng, số phần tử phân biệt có thể bằng K , rồi vượt quá K , rồi tăng tiếp, khiến con trỏ L không thể di chuyển một chiều thuần túy.

Giải pháp đột phá:

Ta sử dụng hệ thức bù trừ:

$$\text{Count}(\text{chính xác } \mathbf{K}) = \text{Count}(\leq \mathbf{K}) - \text{Count}(\leq \mathbf{K} - 1)$$

- Bài toán $\text{Count}(\leq K)$ ("Đếm số đoạn con có **nhều nhất** K phần tử phân biệt") có **tính đơn điệu tuyệt đối**: Khi mở rộng R , số phần tử phân biệt chỉ có tăng hoặc giữ nguyên. Ta giải nó bằng Hai con trỏ trong $O(N)$.
- Tương tự, bài toán $\text{Count}(\leq K - 1)$ cũng giải bằng Hai con trỏ trong $O(N)$.
- Lấy hiệu của hai kết quả, ta giải được bài toán "chính xác K " trong tổng thời gian $O(N) + O(N) = O(N)!$

4. Tài liệu tham khảo & Bài giảng đề xuất (References)

4.1. Giáo trình & Sách thuật toán kinh điển

1. **Competitive Programmer's Handbook** — *Antti Laaksonen*:
2. [Chapter 8: Amortized Analysis \(Section 8.1: Two Pointers Method & Subarray Sum\)](#).
3. **Introduction to Algorithms (CLRS 4th Edition)** — *Thomas H. Cormen et al.*:
4. *Chapter 2: Getting Started (Merge Sort step & Two-pointer scanning)*.
5. *Chapter 17: Amortized Analysis (Aggregate analysis of Two Pointers)*.
6. **Competitive Programming 4 (CP4 - Book 1)** — *Steven Halim*:
7. *Section 3.2: Complete Search & Two Pointers Speedups*.

4.2. Bài giảng & Chuyên đề trực tuyến (Online Resources & Wikis)

1. **USACO Guide (Silver Section)**:
2. [Two Pointers Technique Guide](#) — Hướng dẫn phương pháp tư duy, trực quan hóa và bài tập mẫu.
3. **VNOI Wiki**:
4. [Kỹ thuật Hai con trỏ \(Two Pointers\)](#) — Cẩm nang toàn diện về 2 con trỏ cùng chiều và ngược chiều.
5. **Codeforces Edu Course**:
6. [Two Pointers Method \(ITMO Academy: Pilot Course\)](#) — Khóa học chính thức của ĐH ITMO với hàng chục bài tập thực hành hai con trỏ có phân cấp từ cơ bản đến nâng cao.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.